

CSC-4MI04 - Reconnaissance d'Images

TP3 : Classification d'images par CNN

QUINTELLA PINTO Alfredo

GOMES BIAGGI Naomy

MOCZYDLOWER Carolina

Encadrant :

MANZANERA Antoine

HARIAT Marwane

Introduction

Ce travail pratique porte sur l'expérimentation des réseaux de neurones convolutifs (CNN) pour la classification d'images. Nous utilisons l'API Keras avec le backend TensorFlow pour concevoir, entraîner et évaluer des architectures neuronales sur la base de données CIFAR10, qui contient 60 000 images couleur de 32×32 pixels réparties en 10 classes (*plane, car, bird, cat, deer, dog, frog, horse, ship, truck*).

L'objectif est de comprendre la structure des réseaux de convolution, la dynamique de l'apprentissage, et l'influence des différents hyperparamètres sur les performances d'un modèle. Nous explorons également les phénomènes de sur-apprentissage et les mécanismes pour les atténuer, ainsi que la visualisation des cartes d'activation pour interpréter le comportement interne du réseau.

Prérequis et environnement de travail

Les codes utilisés dans ce travail sont basés sur ceux mis à disposition sur ([Lien Codes](#)). L'ensemble des contenus produits par notre groupe se trouve dans le dépôt Github ([Lien Github](#)).

Structuration des données

Question 2.1

Pourquoi divise-t-on la base en trois sous-ensembles $\{training, validation, test\}$?

La division en trois sous-ensembles répond à un besoin fondamental de généralisation et d'évaluation honnête du modèle :

- **Training (entraînement)** : c'est sur cet ensemble que le modèle ajuste ses paramètres (poids et biais) par rétropropagation du gradient. Dans notre code, `n_training_samples = 5000` images sont utilisées.
- **Validation** : cet ensemble, non utilisé pour la mise à jour des poids, sert à surveiller les performances du modèle *pendant* l'entraînement. Il permet de détecter le sur-apprentissage et de sélectionner le meilleur modèle via `ModelCheckpoint`.
- **Test** : cet ensemble est utilisé *une seule fois*, après la fin de l'entraînement, pour estimer la performance réelle du modèle sur des données jamais vues. Il garantit une évaluation non biaisée par les choix d'hyperparamètres.

Utiliser le même ensemble pour l'entraînement et l'évaluation conduirait à surestimer les performances du modèle, car il aurait simplement mémorisé les données d'entraînement.

Question 2.2

Que réalise la fonction `standardize` ? Quel est son intérêt ?

La fonction `standardize` centre et réduit les valeurs de chaque image *individuellement* :

$$x' = \frac{x - \mu_{\text{img}}}{\sigma_{\text{img}}}$$

où μ_{img} et σ_{img} sont la moyenne et l'écart-type calculés sur tous les pixels de l'image. Après transformation, chaque image a une moyenne nulle et un écart-type unitaire.

Son intérêt est double : elle **normalise l'échelle** des entrées (sans standardisation, les valeurs de pixels entre 0 et 255 ralentissent la convergence du gradient), et elle **réduit l'effet de l'éclairage** en atténuant les variations globales de luminosité entre images.

Architecture du réseau

Question 3.1

Dans le résumé de l'architecture du réseau qui s'affiche dans la partie II.3, que sont les *trainable params* ? Comment leur nombre est-il calculé ?

Les *trainable params* sont les paramètres ajustés pendant l'entraînement par rétropropagation — c'est-à-dire les **poids** et les **biais** de chaque couche apprenante. Pour l'architecture de base :

- **Conv2D** (8 filtres, noyau 3×3 , entrée $32 \times 32 \times 3$) : $8 \times (3 \times 3 \times 3 + 1) = \mathbf{224}$ paramètres.
- **MaxPool2D, Flatten** : aucun paramètre appris. La sortie de MaxPool est $16 \times 16 \times 8 = 2048$ neurones.
- **Dense(64)** : $2048 \times 64 + 64 = \mathbf{131\ 136}$ paramètres.
- **Dense(10)** : $64 \times 10 + 10 = \mathbf{650}$ paramètres.

Total : $224 + 131\ 136 + 650 = \mathbf{132\ 010}$ paramètres entraînaibles, confirmé par `model.summary()`. La grande majorité provient de la couche Dense, ce qui illustre l'inconvénient du **Flatten** directement après convolution : le nombre de connexions explose lorsque la carte de features est grande.

Apprentissage

Question 4.1

À quoi correspondent une époque (*epoch*), une étape (*step*) et un lot (*batch*) ?

- **Lot (*batch*)** : sous-ensemble d'images présenté simultanément au réseau à chaque étape. Avec `batch_size=32`, le gradient est calculé et mis à jour une fois par lot, offrant un compromis entre stabilité et vitesse de convergence.
- **Étape (*step*)** : traitement d'un lot et mise à jour des poids associée. Pour 5000 images et `batch_size=32` : $\lceil 5000/32 \rceil = 157$ étapes par époque.
- **Époque (*epoch*)** : passage complet sur l'ensemble de la base d'entraînement. Après chaque époque, les métriques de validation sont calculées et les callbacks exécutés. Avec `epochs=20`, le réseau parcourt 20 fois les 5000 images d'entraînement.

Question 4.2

Essayez d'ajuster la taille de lot (*batchsize*), et analysez son effet sur le temps de calcul d'une étape, d'une époque, ainsi que sur les courbes d'apprentissage.

Nous avons évalué `batch_size` $\in \{8, 32, 128\}$ avec l'optimiseur SGD ($lr=0.01$, sans momentum) sur 20 époques.

<code>batch_size</code>	Steps/epoch	Temps/epoch (s)	Temps total (s)	Val acc finale
8	625	1.8	36.5	0.451
32	156	0.7	13.8	0.401
128	39	0.4	8.1	0.296

TABLE 1 – Effet du `batch_size` sur le temps de calcul et la précision de validation (SGD, 20 époques).

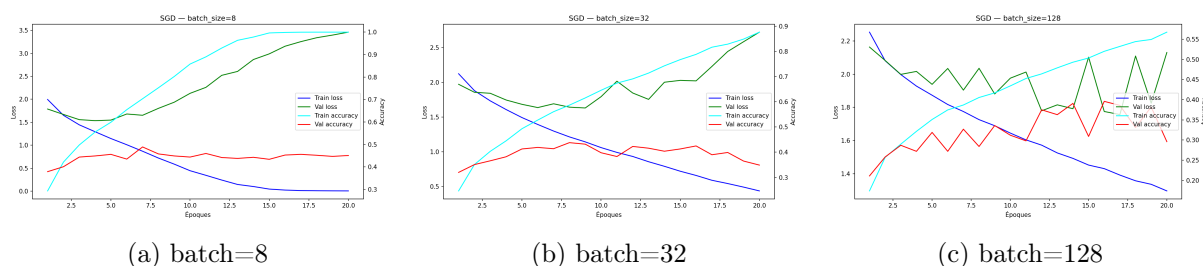


FIGURE 1 – Courbes d'apprentissage pour différentes valeurs de `batch_size` (SGD, $lr=0.01$, 20 époques).

Effet sur le temps de calcul. Le temps par époque diminue avec l'augmentation du `batch_size` : de 1.8s pour `batch=8` à 0.4s pour `batch=128`, car le nombre d'étapes passe de 625 à 39.

Effet sur les courbes. Pour `batch=8`, la training accuracy atteint 1.0 (sur-apprentissage sévère) tandis que la validation accuracy stagne à 0.45. La validation loss diverge fortement, car chaque mise à jour est basée sur un gradient très bruité. Pour `batch=128`, les courbes sont plus lisses mais la convergence est plus lente et la précision de validation finale plus faible (0.296).

Compromis optimal. Le `batch_size=32` offre le meilleur équilibre entre vitesse, stabilité et précision (0.401), et est retenu pour la suite des expériences.

Question 4.3

Comparez les différentes fonctions d'optimisation : *SGD*, *SGD avec Momentum*, et *Adam*.

Nous avons comparé les trois optimiseurs avec `batch_size=32` fixé, en réinitialisant les poids avant chaque expérience.

Optimiseur	Paramètres	Temps total (s)	Val acc finale
SGD	$lr=0.01$, $\mu=0.0$	13.6	0.349
SGD + Momentum	$lr=0.01$, $\mu=0.9$	16.3	0.448
Adam	$lr=0.001$	14.4	0.448

TABLE 2 – Comparaison des optimiseurs (`batch_size=32`, 20 époques).

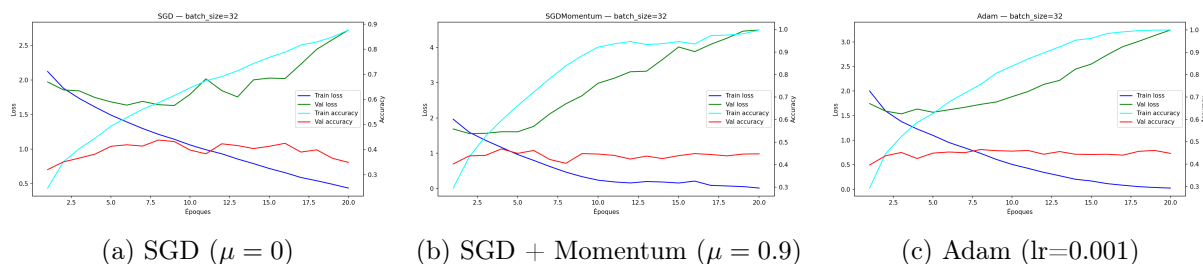


FIGURE 2 – Courbes d'apprentissage pour les trois optimiseurs (`batch_size=32`, 20 époques).

SGD (sans momentum). Convergence régulière mais lente. Val acc de seulement 0.349, avec un sur-apprentissage progressif visible dès l'époque 5 (training accuracy proche de 1.0, validation accuracy stagnante).

SGD avec Momentum ($\mu = 0.9$). L'accumulation d'inertie dans la direction du gradient ($v \leftarrow \mu v - \eta \nabla \mathcal{L}$) accélère nettement la convergence en début d'entraînement. Val acc de 0.448, soit +0.10 par rapport au SGD pur. Cependant, le momentum amplifie aussi le sur-apprentissage (val loss diverge rapidement).

Adam. Combine un momentum sur le gradient et une adaptation du taux d'apprentissage par paramètre. Il atteint la même val acc (0.448) que SGD+Momentum mais avec des courbes plus stables. Adam se distingue par sa robustesse : ses performances varient peu entre différentes configurations, ce qui en fait l'optimiseur le plus polyvalent.

Hyperparamètres

Question 5.1

Identifiez dans l'ensemble du code les hyperparamètres du modèle, précisez pour chacun sa nature et l'influence qu'il peut avoir dans le processus d'apprentissage.

Les hyperparamètres sont les paramètres fixés *avant* l'entraînement, non appris par rétropropagation, qui conditionnent la structure et la dynamique du modèle.

Hyperparamètre	Valeur	Influence
<code>n_training_samples</code>	5000	Taille de la base d'entraînement. Plus elle est grande, meilleure est la généralisation, mais plus l'entraînement est lent.
<code>filters</code> (Conv2D)	8	Nombre de filtres. Détermine la richesse de la représentation extraite. Trop peu : sous-apprentissage. Trop : surcoût computationnel.
<code>kernel_size</code>	(3,3)	Taille du noyau. Un noyau plus grand capture des motifs plus larges mais augmente le nombre de paramètres.
<code>kernel_regularizer</code> (L2)	0.00	Pénalise les grands poids pour réduire le sur-apprentissage.
<code>Dropout</code> (taux)	0.0	Désactive aléatoirement des neurones pendant l'entraînement, forçant des représentations redondantes et réduisant le sur-apprentissage.
<code>pool_size</code>	(2,2)	Facteur de sous-échantillonnage. Réduit la dimension spatiale et introduit une invariance locale aux translations.
<code>units</code> (Dense)	64	Capacité du classifieur final.
<code>learning_rate</code>	0.01	Taux d'apprentissage. Trop élevé : divergence. Trop faible : convergence lente. Hyperparamètre le plus sensible.
<code>momentum</code>	0.0	Terme d'inertie du SGD. Accélère la convergence dans les directions persistantes du gradient.
<code>batch_size</code>	32	Influe sur la stabilité du gradient, le temps de calcul et le risque de sur-apprentissage (cf. Q4.2).
<code>epochs</code>	20	Nombre de passages sur la base. Trop peu : sous-apprentissage. Trop : sur-apprentissage sans arrêt précoce.

TABLE 3 – Hyperparamètres du modèle identifiés dans le code.

Approfondissement du modèle

Question 6.1

Essayez d'améliorer les performances de votre modèle en approfondissant le réseau à plusieurs couches de convolution.

Pour améliorer les performances du modèle de base (val acc ≈ 0.44), nous avons conçu une architecture plus profonde composée de trois blocs convolutifs successifs, chacun suivi d'une normalisation par lots (`BatchNormalization`), d'un sous-échantillonnage (`MaxPool2D`) et d'une régularisation par abandon (`Dropout`). Le classifieur final utilise un `GlobalAveragePooling2D` à la place du `Flatten`, ce qui réduit drastiquement le nombre de paramètres tout en préservant

l'information spatiale.

Question 6.2

Représentez graphiquement le réseau multi-couches final, récapitulez ses hyperparamètres, affichez les courbes et les matrices de confusion correspondantes.

Architecture du modèle profond

Couche	Type	Output shape	Paramètres
—	Input	(32, 32, 3)	0
4*Bloc 1	Conv2D(16, 3×3, relu)	(32, 32, 16)	448
	BatchNormalization	(32, 32, 16)	64
	Conv2D(16, 3×3, relu)	(32, 32, 16)	2 320
	MaxPool2D(2×2) + Dropout(0.25)	(16, 16, 16)	0
4*Bloc 2	Conv2D(32, 3×3, relu)	(16, 16, 32)	4 640
	BatchNormalization	(16, 16, 32)	128
	Conv2D(32, 3×3, relu)	(16, 16, 32)	9 248
	MaxPool2D(2×2) + Dropout(0.25)	(8, 8, 32)	0
3*Bloc 3	Conv2D(64, 3×3, relu)	(8, 8, 64)	18 496
	BatchNormalization	(8, 8, 64)	256
	MaxPool2D(2×2) + Dropout(0.30)	(4, 4, 64)	0
3*Classifieur	GlobalAveragePooling2D	(64)	0
	Dense(64, relu) + Dropout(0.40)	(64)	4 160
	Dense(10, softmax)	(10)	650
Total paramètres entraînables			40 186

TABLE 4 – Architecture du modèle profond. Les paramètres non entraînables (224) correspondent aux moyennes mobiles des couches `BatchNormalization`.

Courbes d'apprentissage

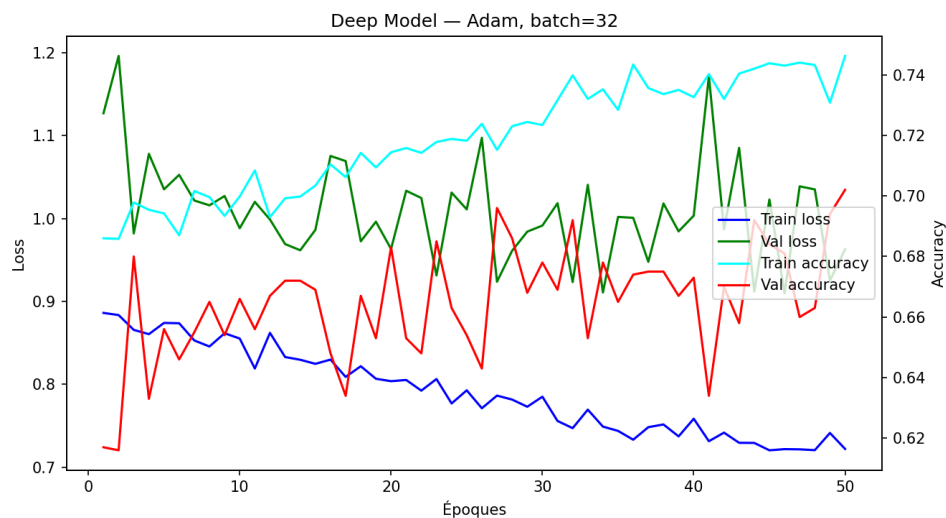


FIGURE 3 – Courbes d'apprentissage du modèle profond (Adam, $lr=0.001$, $batch_size=32$, 50 époques).

Performances

Ensemble	Accuracy	Loss
Train	0.737	0.719
Validation	0.628	1.112
Test	0.592	1.291

TABLE 5 – Performances finales du modèle profond.

Classe	Précision (%)
plane	41.75
car	65.74
bird	40.40
cat	30.77
deer	38.83
dog	49.53
frog	90.91
horse	60.00
ship	79.38
truck	90.00
Global	59.20

TABLE 6 – Précision par classe sur les 1000 images de test.

Question 6.3

Critiquez votre modèle en soulignant ses avantages et limitations.

Avantages. Le modèle profond améliore significativement les performances par rapport au modèle de base : la précision de test passe de $\approx 44\%$ à 59.2% . L'utilisation du **GlobalAveragePooling2D** à la place du **Flatten** réduit le nombre de paramètres de 132 010 à 40 186, soit une réduction de 70% , tout en améliorant la précision. Les couches **BatchNormalization** stabilisent l'apprentissage et permettent d'utiliser des taux d'apprentissage plus élevés. Le **Dropout** à chaque bloc limite efficacement le sur-apprentissage.

Limitations. Un écart notable subsiste entre train (73.7%) et test (59.2%), indiquant un sur-apprentissage résiduel. Les classes *cat* (30.77%) et *bird* (40.40%) restent difficiles à classer, probablement en raison de leur variabilité visuelle élevée et de leur ressemblance avec d'autres classes (*dog*, *deer*). La taille de la base d'entraînement (5000 images) constitue le principal facteur limitant : des modèles similaires entraînés sur l'intégralité de CIFAR10 (50 000 images) atteignent typiquement $75\text{--}85\%$.

Sur-apprentissage

Question 7.1

Comment pouvez-vous observer le phénomène de sur-apprentissage (*overfitting*) sur les courbes ? Montrez et commentez un exemple.

Le sur-apprentissage se manifeste sur les courbes d'apprentissage par deux symptômes caractéristiques et simultanés :

- La **training loss** continue de diminuer (et la training accuracy d'augmenter) d'époque en époque.
- La **validation loss** cesse de diminuer et commence à *augmenter*, tandis que la validation accuracy stagne ou décroît légèrement.

Cet écart croissant entre les courbes d'entraînement et de validation indique que le modèle mémorise les données d'entraînement au lieu d'apprendre des caractéristiques généralisables.

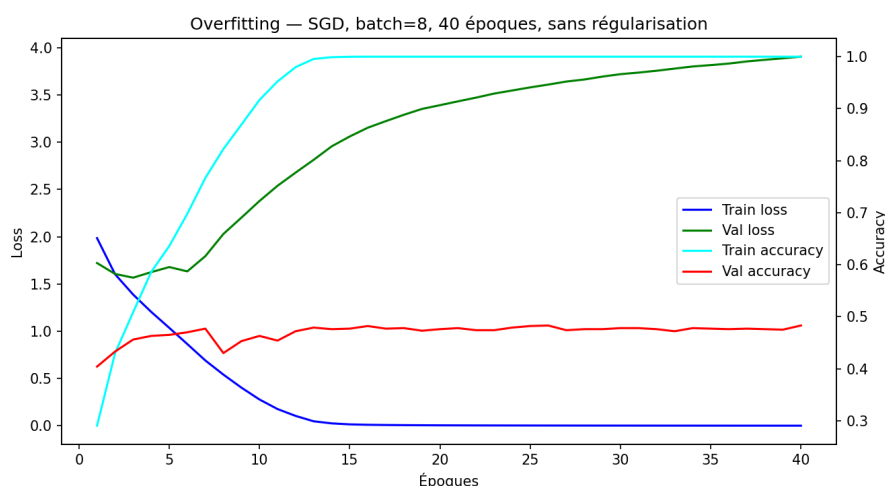
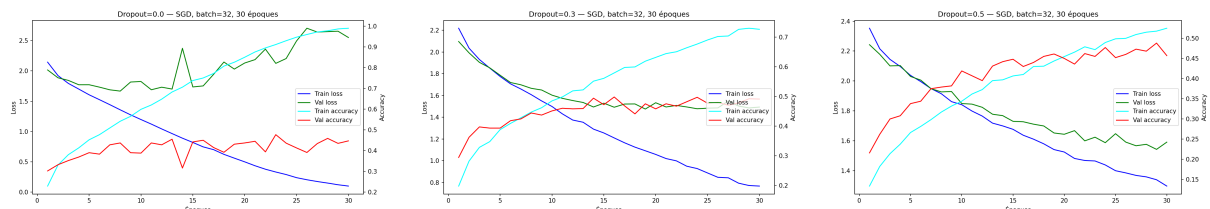


FIGURE 4 – Exemple de sur-apprentissage sévère : SGD, **batch=8**, 40 époques, sans régularisation. La training accuracy atteint 1.0 dès l'époque 15, tandis que la validation accuracy stagne à ≈ 0.47 et que la validation loss diverge jusqu'à 4.0.

Question 7.2

Quels mécanismes peuvent influencer favorablement sur ce phénomène? Montrez des exemples.

Nous avons expérimenté l'effet du **Dropout** comme mécanisme de régularisation. Le Dropout désactive aléatoirement une fraction p des neurones pendant l'entraînement, forçant le réseau à apprendre des représentations redondantes et réduisant la co-adaptation des neurones.



(a) Dropout = 0.0 (val acc = 0.447) (b) Dropout = 0.3 (val acc = 0.491) (c) Dropout = 0.5 (val acc = 0.457)

FIGURE 5 – Effet du taux de Dropout sur l'apprentissage (SGD, batch=32, 30 époques).

Sans Dropout (0.0), le sur-apprentissage est visible : la training accuracy atteint 1.0 tandis que la validation loss diverge. Avec **Dropout=0.3**, l'écart entre les courbes d'entraînement et de validation se réduit nettement, et la val acc atteint son maximum (0.491). Les courbes de validation loss et d'accuracy convergent davantage, signe d'une meilleure généralisation. Avec **Dropout=0.5**, le taux trop élevé ralentit excessivement l'apprentissage (val acc redescend à 0.457) : trop de neurones sont désactivés simultanément, ce qui empêche le modèle d'apprendre des représentations suffisamment riches. Un taux de **Dropout=0.3** est donc **optimal** pour ce modèle et ce dataset.

D'autres mécanismes peuvent également réduire le sur-apprentissage : la **régularisation L2** (pénalisation des poids de grande amplitude via `kernel_regularizer=12(λ)`), la **Batch Normalization** (stabilise les distributions d'activations entre couches), et l'**augmentation de données** (*data augmentation* : rotations, flips, translations aléatoires des images d'entraînement).

Cartes d'activation

Question 8.1

Comment pouvez-vous interpréter ces cartes d'activation pour les features de la première couche ?

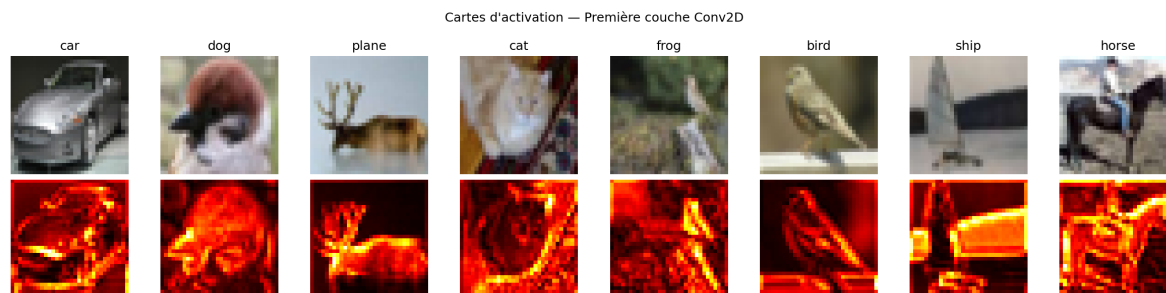


FIGURE 6 – Cartes d'activation de la première couche Conv2D du modèle profond (somme des activations positives sur les 16 filtres). Ligne du haut : images originales avec leur classe prédite. Ligne du bas : cartes d'activation correspondantes (colormap *hot*).

Les cartes d'activation de la première couche Conv2D révèlent les régions de l'image qui activent le plus fortement les filtres appris. À ce niveau superficiel, les filtres répondent principalement aux **contours et aux transitions de luminosité** : on observe des zones d'activation élevée (blanc/jaune) le long des bords des objets (contour du corps de l'animal, bords du véhicule, horizon entre ciel et sol), tandis que les zones uniformes (fond, arrière-plan homogène) produisent peu d'activation.

Ce comportement est caractéristique des premières couches des CNN : elles jouent un rôle similaire aux détecteurs de contours classiques (Sobel, Canny), mais de manière apprise et spécialisée selon les données d'entraînement. La résolution spatiale est préservée (32×32), car `padding='same'` est utilisé.

Question 8.2

Une fois le modèle plus profond obtenu, visualisez des cartes d'activation correspondant à des couches plus profondes et interprétez-les.

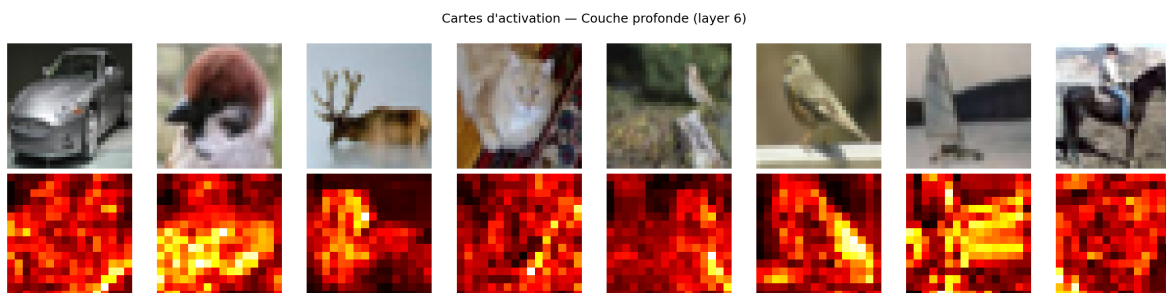


FIGURE 7 – Cartes d'activation d'une couche profonde du modèle (layer 6 — deuxième bloc convolutif, $16 \times 16 \times 32$). Même disposition que la figure précédente.

Les cartes d'activation de la couche profonde (Bloc 2, $16 \times 16 \times 32$) présentent un comportement qualitativement différent de celles de la première couche. La résolution spatiale est réduite de moitié (16×16 au lieu de 32×32) suite au MaxPool2D du Bloc 1, ce qui confère une invariance locale aux petites translations. Les zones d'activation sont **plus diffuses et moins directement liées aux contours** : elles tendent à correspondre à des *régions sémantiques* de l'objet (partie centrale du corps de l'animal, zone d'intérêt principale de la scène) plutôt qu'à des détails de bas niveau.

Ce phénomène illustre le principe fondamental des CNN profonds : les premières couches extraient des caractéristiques locales de bas niveau (contours, textures), tandis que les couches profondes

combinent ces primitives pour détecter des **motifs de plus haut niveau** (formes, parties d'objets). L'information spatiale précise est progressivement sacrifiée au profit d'une représentation plus abstraite et invariante.

Conclusion

Ce travail pratique avait pour objectif d'expérimenter les réseaux de neurones convolutifs pour la classification d'images sur la base CIFAR10, en explorant progressivement l'architecture, les hyperparamètres, les phénomènes de sur-apprentissage et la visualisation des représentations internes.

L'étude de l'effet du **batchsize** a mis en évidence un compromis fondamental : les petits lots (batch=8) produisent des gradients bruités qui favorisent le sur-apprentissage malgré une meilleure précision de validation, tandis que les grands lots (batch=128) offrent des courbes plus stables mais une convergence plus lente et des performances inférieures. Le batch=32 constitue le meilleur compromis pour ce dataset. La comparaison des **optimiseurs** a montré que SGD+Momentum et Adam surpassent nettement le SGD pur (+0.10 de val acc), Adam se distinguant par sa robustesse aux différentes configurations.

L'**approfondissement du modèle** a constitué l'amélioration la plus significative : en passant d'un réseau à une seule couche convolutive (132 010 paramètres, val acc $\approx 44\%$) à une architecture à trois blocs convolutifs avec BatchNormalization et GlobalAveragePooling2D (40 186 paramètres, test acc = 59.2%), nous avons obtenu une amélioration de +15 points de précision tout en réduisant le nombre de paramètres de 70 %. Ce résultat illustre l'importance de la profondeur et de la structure hiérarchique dans les CNN : les premières couches extraient des primitives visuelles (contours, textures), tandis que les couches profondes combinent ces primitives en représentations plus abstraites, comme le confirment les **cartes d'activation**.

L'étude du **sur-apprentissage** a montré que le Dropout constitue un mécanisme efficace de régularisation, avec un taux optimal de 0.3 pour ce modèle (+0.044 de val acc par rapport à l'absence de régularisation). Un taux trop élevé (0.5) nuit à l'apprentissage en désactivant trop de neurones simultanément.

Les principales limitations observées sont l'écart persistant entre train (73.7 %) et test (59.2 %), signe d'un sur-apprentissage résiduel, et la difficulté à classer correctement certaines classes visuellement proches (*cat* : 30.77 %, *bird* : 40.40 %). Ces limitations sont principalement imputables à la taille restreinte de la base d'entraînement (5 000 images). Des pistes d'amélioration incluent l'augmentation de données (*data augmentation*), l'utilisation de l'intégralité du dataset CIFAR10, ou le recours au *transfer learning* à partir de modèles pré-entraînés sur ImageNet.

Références

- [1] Manzanera, A. (2026). *CSC-4MI04 - Reconnaissance d'Images - Supports de cours*. ENSTA Paris.
- [2] Krizhevsky, A. *Learning Multiple Layers of Features from Tiny Images*. Technical Report, University of Toronto, 2009. ([Disponible ici](#))